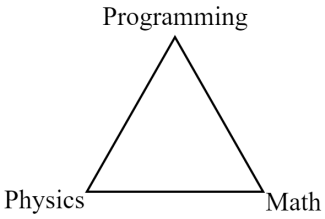
Lecture 5: CPU

Nic Davey & Lui Holder-Pearson

July 30, 2026

Learning Goals

- Understand the subcomponents of a CPU, and what each one does.
- Have an understanding of how addition is implemented within an ALU in logic gates.
- Understand how the addition unit can be used to implement comparison behaviour.



Further Reading

ENEL373
ENCE361

References

Rizzoni and Kearns. *Principals and Applications of Electrical Engineering*, Chapters 13 & 14.

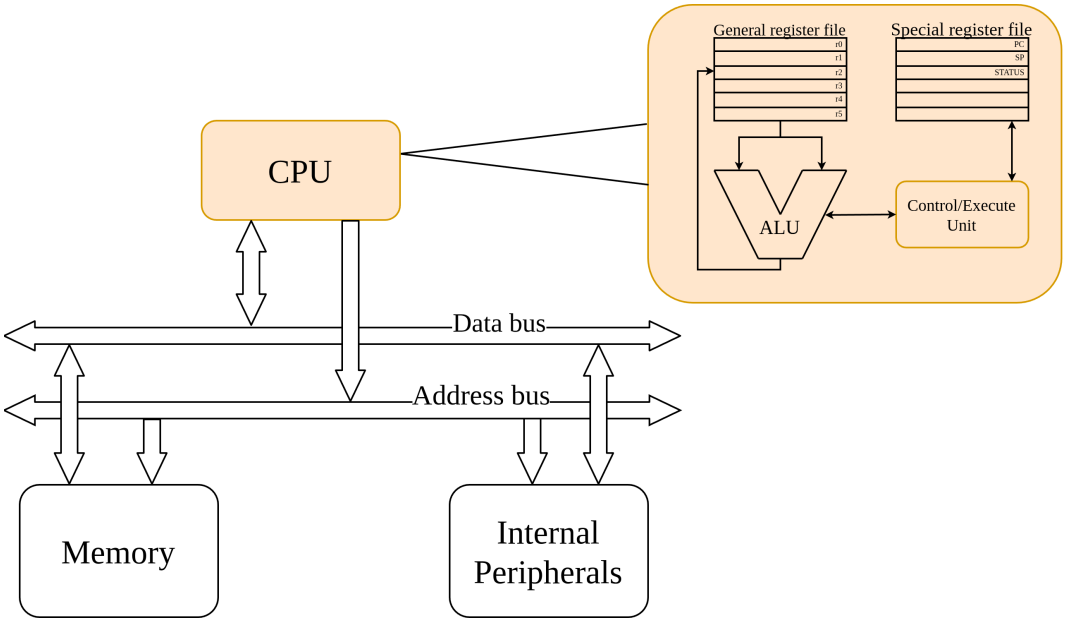


Figure 1: The central processing unit within the wider computer architecture.

The central processing unit (CPU) provides the computer the ability to perform various operations on various operands.

Recall that:

- A computer is comprised parts:
  1. **Memory and buses**: The ability to store data, and to move it around the inside of the computer.
  2. **CPU**: The 'compute' part of computer, that allows calculations and other operations to be performed.
  3. **Internal peripherals**: Dedicated hardware units inside the microcontroller, including timer/counter units, communication units, and GPIO ports.

1 Inside a CPU

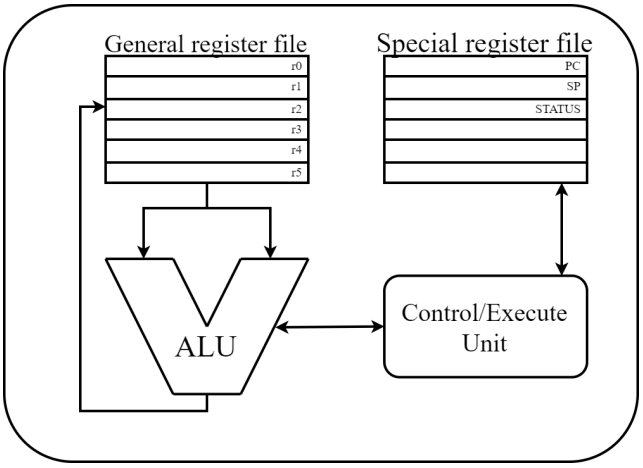


Figure 2: Components of the CPU.

1. **Arithmetic logic unit (ALU)**:

2. **General purpose registers**:

3. **Control/execute unit**:

4. **Control registers**:

1.1 Registers

Both sets of registers are very fast, and very physically close to the ALU to enable fast performance. They are usually very expensive — financially and physically — in that they are comprised a large number of transistors.

The general register layout in the ATMega32u2 is shown in Figure 3.

Figure 6-2. AVR CPU General Purpose Working Registers

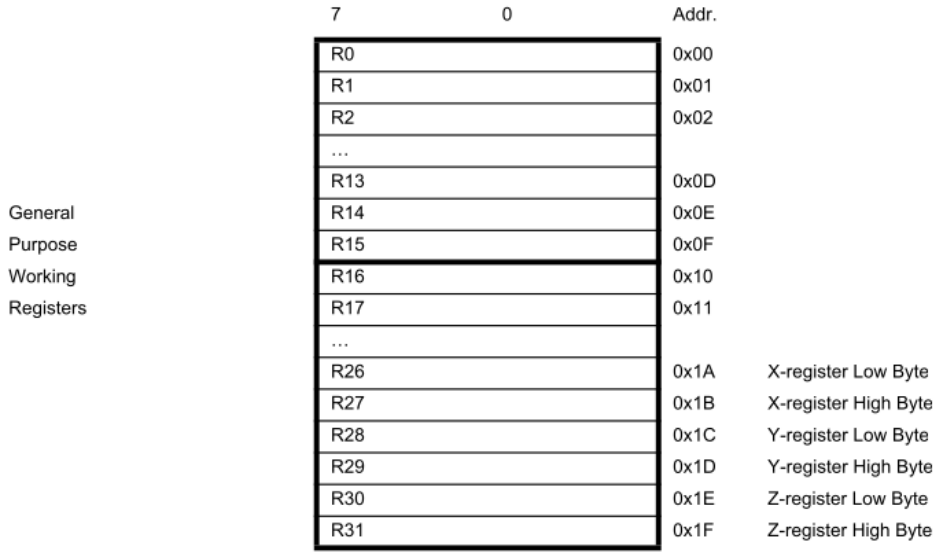


Figure 3: ATMega32u2 general purpose register layout.

The ATMega32u2 control/special register contains a small number of 8-bit registers. The 'status register', a single 8-bit register shown in Figure 4.

Bit	7	6	5	4	3	2	1	0	
0x3F (0x5F)	I	T	H	S	V	N	Z	C	SREG
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Figure 4: ATMega32u2 special purpose register.

The bits of particular interest the special flags used to enable faster decision making about program flow.:

- **C: Carry flag**. This indicates a carry in an arithmetic or logic operation.
- **Z: Zero flag**. This indicates a zero result in an arithmetic or logic operation.
- **N: Negative flag**. This indicates a negative result in an arithmetic or logic operation.

The other registers in the control registers contain:

- **PC: Program counter**: The address of the next instruction in the program memory.
- **SP: Stack pointer**: The address of the "top of the stack", which is used for dynamic variable handling.

The role of the program counter and stack pointer are discussed further in *CA L11 - Execution*, and *CA L12 - Function calls*.

Control/execute unit

The control and execute unit is responsible for controlling the ALU and the flow of data from the registers to the inputs of the ALU, and from the output of the ALU to the registers.

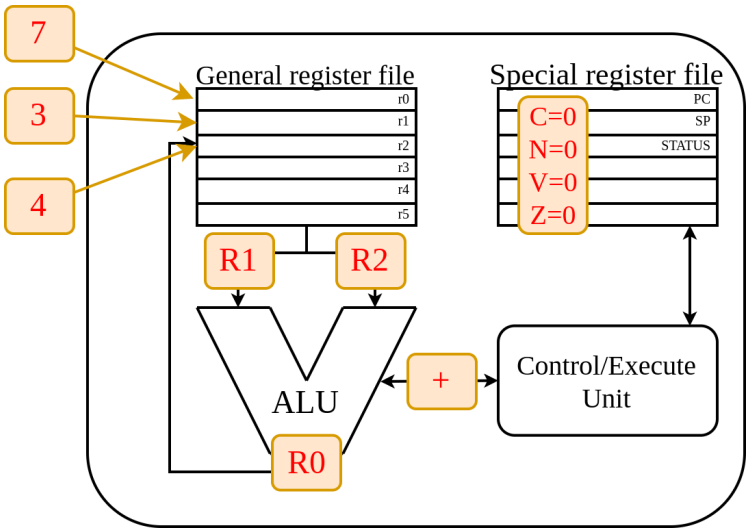
Specifically, the control/execute unit takes the instruction from program memory, and ensures that it is actually carried out in terms of:

1. Loading the *next instruction* from memory and interpreting it as a *finite state machine*
2. Loading the correct variables *into the registers* from memory
3. Loading the variables *into the ALU* from the registers
4. Instructing the ALU *which* operation to perform, and *when*
5. Storing the output of the ALU operation in the correct register.

FSMs are discussed in depth in *CA L9 - Finite state machines*.

An example

```
y = a + b
```



What does a CPU *actually* look like?

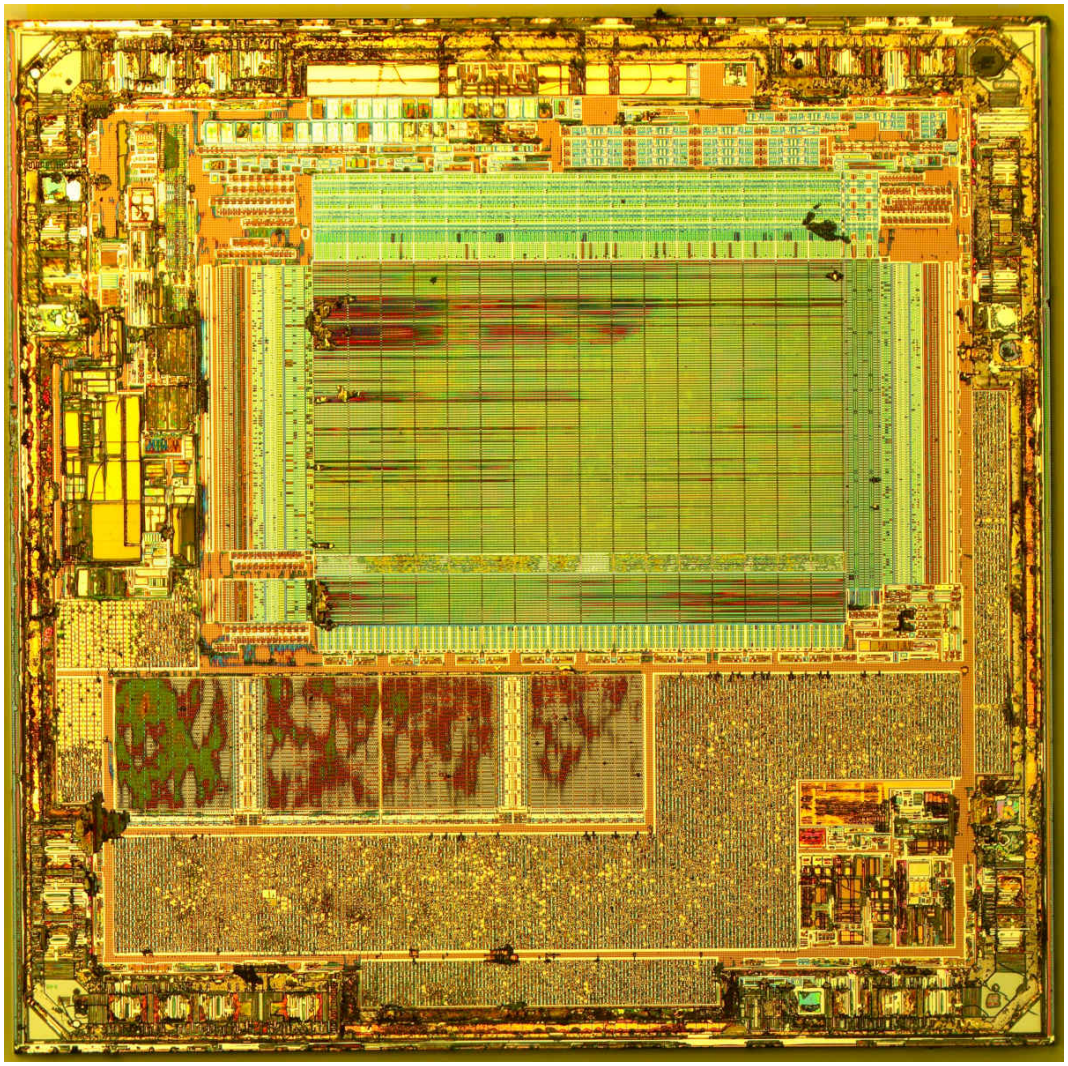
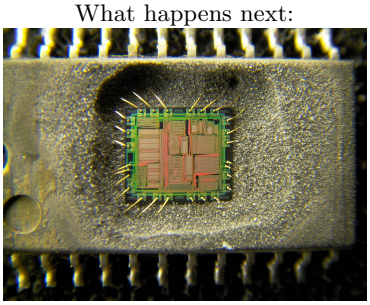


Figure 5: AVR ATMega328p (Arduino Nano microcontroller) 'delidded' (HD zoomable source) — the plastic is eaten way with hydroflouric acid.

What happens next:



5.7 As an example of scale, the ATTiny4 (512B Flash, 32B RAM) has a die size of 1.37 × 0.93mm. It is available in packages 2.9x2.8mm, or 2x2mm.

2 Arithmetic logic unit design

The ALU provides the means to perform arithmetic, comparisons, and logic operations.

Recall: Binary numerical encoding

It is necessary to have a method of encoding numbers within digital systems. Because there are only two states within memory available, `0` or `1`, we are forced to use the binary number system, which is base 2 as opposed to base 10:

$$101_2 = 5_{10}$$

Problems:

$1100_2 =$
$23_{10} =$

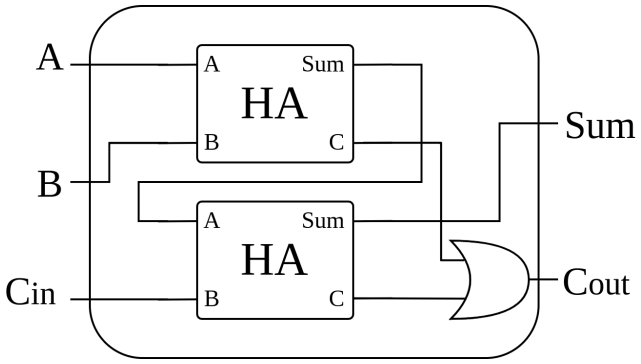
2.1 Adders

Again, we need to build our system out of logic gates, as these are the fundamental components that we can physically make with transistors.

A	B	$A + B_{(math)}$	C	S

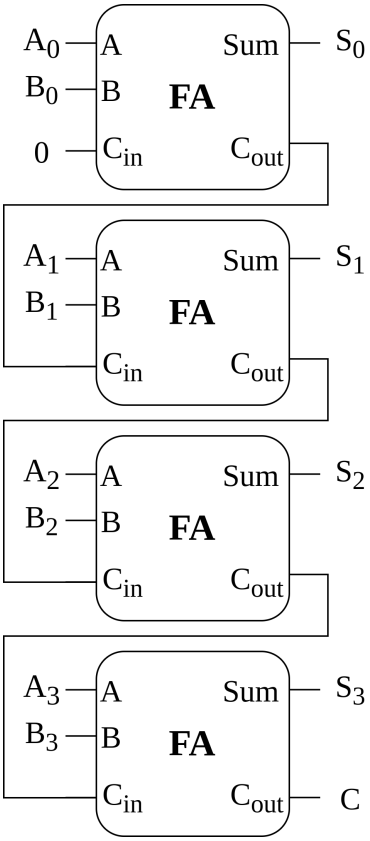
We can therefore write the logic expressions, and corresponding logic circuit for a **half adder**:

A full adder is the combination of two half adders to allow for an incoming carry bit:



And finally by cascading full adders we can create the complete addition aspect of an ALU:

$$\begin{array}{cccccc} & & c_2 & c_1 & c_0 & \\ & & A_3 & A_2 & A_1 & A_0 \\ + & & B_3 & B_2 & B_1 & B_0 \\ \hline & C_{(3)} & S_3 & S_2 & S_1 & S_0 \end{array}$$

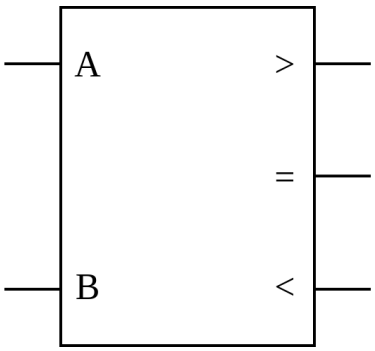


2.2 Comparisons

The addition circuit in the ALU can implement a wide variety of functionality. It would be possible to implement a specific circuit for comparing values to see if they are equal, or which is greater than or less than. for example:

A	B	gt	eq	lt
00	00			
00	01			
00	10			
00	11			
01	00			
01	01			
01	10			
01	11			

... etc ...



The three functions for greater than, less than, and equal could then be reduced to the simplest form (which will absolutely not be simple), and implemented. However, we can instead make use of the combination of:

- The addition unit
- The ease of two's complement addition / subtraction
- The flags

3 Tutorial questions

1. Write an expression for a full adder's C_{out} in terms of the inputs A, B, and C_{in}.
2. Draw a circuit using AND, OR, and NOT logic gates that implements your expression for C_{out}.
3. Consider a comparator that operates on two two-bit numbers: $A = A_1A_0$, and $B = B_1B_0$:
   (a) Write a logic expression for the gt output of this comparator. Your expression should use A_0, A_1, B_0, and B_1 as inputs.
   (b) Draw a logic gate diagram based on your logic expression.
   (c) Imagine how incredibly complex an 8-bit comparator would be using the same design process.

Summary

- Understand the subcomponents of a CPU, and what each one does.
  → **ALU**: actually performs the operations required, and in doing so generates the data that can be used in subsequent operations.
  → **General purpose registers**: Physically close to the ALU, provides memory for the inputs to, and outputs from the ALU.
  → **Control/execute unit**: Responsible for controlling the data flow to and from the general purpose registers, and acts as a finite state machine in decoding the instructions from the program memory.
  → **Control registers**: Stores metadata about the latest calculations.
- Have an understanding of how the adder of an ALU is formed with logic gates.
  → **Half adders** are used to compare two individual bits, with outputs `sum`, and `carry`.
  → **Full adders** use two half adders cascaded. Full adders themselves are cascaded to form a complete n-bit addition unit.
- Understand how the addition unit can be used to implement comparison behaviour.
  → **Addition and subtraction** can be implemented with the same addition circuit, provided two's complement notation is used for negative numbers.
  → **Comparisons** Can be achieved by using the **Z**ero and **N**egative flags to determine which value is larger.

Next Lecture: Lecture 6 - Combinational logic circuits

4 Questions?